



AQI Forecasting System

Technical Project Report

Date: 3 June 2026

Prepared by: Muhammad Wasif Shakeel

City: Rawalpindi

Table of Contents

- Table of Contents.....2**
- 1. Project Overview.....3
 - 1.1 Goals.....3
 - 1.2 Technology Stack.....3
- 2. System Architecture.....4**
 - 2.1 Pipeline Stages.....4
 - 2.2 Automation.....4
- 3. Data and Feature Engineering.....5**
 - 3.1 Data Source.....5
 - 3.2 Feature Catalog.....5
 - 3.3 Data Quality Handling.....5
- 4. Exploratory Data Analysis.....6
 - 4.1 Dataset Overview.....6
 - 4.2 AQI Distribution.....6
 - 4.3 Temporal Coverage and Gap Analysis.....6
 - 4.4 Monthly Average AQI Trend.....7
 - 4.5 Feature Correlation with European AQI.....7
 - 4.6 Permutation Feature Importance.....8
 - 4.7 Naive Baseline Comparison.....9
 - 4.8 Training Window Study.....10
 - 4.9 Lag and Rolling Feature Null Rates.....11
 - 4.10 Training Setups.....11
- 5. Modelling Approach.....12**
 - 5.1 Forecast Formulation.....12
 - 5.2 Model Candidates.....12
 - 5.3 Evaluation.....12
- 6. Evaluation Results.....13**
 - 6.1 Model Comparison.....13
- 7. The Overfitting Challenge - Deep Dive.....14**
 - 7.1 Symptoms.....14
 - 7.2 Attempted Remedies and Observations.....14
 - 7.3 Other Challenges.....14
- 8. What Is Working Well.....16**
 - 8.1 Pipeline Reliability.....16
 - 8.2 Model Registry and Artefacts.....16
 - 8.3 Explainability (SHAP).....16
 - 8.4 Dashboard.....16
- 9. Potential Improvements.....17
 - 9.1 Closing the Generalisation Gaps.....17
 - 9.2 Infrastructure.....17
- 10. Conclusion.....18**

1. Project Overview

This project delivers a production-grade AQI forecasting system for Rawalpindi, combining automated data ingestion, feature engineering, model training, and a web-based dashboard for real-time forecast monitoring.

1.1 Goals

- Produce reliable 72-hour AQI forecasts with transparent evaluation metrics.
- Maintain a reproducible feature pipeline and model registry to track best-performing models over time.
- Expose results through a Next.js dashboard and REST API routes for easy consumption.
- Support full automation with minimal infrastructure overhead using GitHub Actions.

1.2 Technology Stack

Data Source	Open-Meteo API (weather + air quality, hourly)
Storage	MongoDB - feature store, predictions, metrics, EDA summaries
ML Libraries	scikit-learn, LightGBM, XGBoost, CatBoost, TensorFlow
Registry	DagsHub MLflow (REST) - model artefacts and versioning
Orchestration	GitHub Actions - daily pipeline automation
Frontend	Next.js 14, React 18, TypeScript

2. System Architecture

The system follows a layered pipeline architecture. Data flows from ingestion through feature engineering into MongoDB, then into the training and prediction layer, with results surfaced via the Next.js frontend.

2.1 Pipeline Stages

- Ingestion: Open-Meteo is queried for historical and forecast weather/AQI data for Rawalpindi coordinates (33.6007°N, 73.0679°E).
- Feature Engineering: Time features, lag features (1h to 168h), rolling statistics (mean, std, min), weather-derived features, pollutant indices, and spatial constants are computed.
- Feature Store: Processed features are upserted into the `aqi_features_rawalpindi` MongoDB collection using batched `bulk_write` operations.
- Training Pipeline: Loads the last 90 days of features, trains six tree-based models, evaluates all by RMSE, and registers the best as champion.
- Prediction Store: 72-hour ahead predictions with confidence bands are written to `aqi_predictions_rawalpindi`.
- Dashboard: Next.js API routes read from MongoDB and serve the React frontend.

2.2 Automation

All pipelines run daily via GitHub Actions. The feature pipeline detects the latest timestamp in MongoDB and fetches only the overlapping window (last 168 hours), preventing duplicate ingestion. Training skips automatically if no new data has arrived since the last run, keeping compute costs low.

3. Data and Feature Engineering

3.1 Data Source

Open-Meteo provides 30+ hourly weather variables (temperature, humidity, wind, radiation, pressure, precipitation) and 23 air-quality variables (PM2.5, PM10, NO₂, O₃, SO₂, European AQI sub-indices). Historical data goes back several years; the forecast endpoint provides 3-day-ahead signals used during the live feature pipeline run.

3.2 Feature Catalog

A curated TOP_FEATURES set drives both the feature pipeline and the training pipeline, ensuring consistency. Key feature groups include:

- Temporal: hour_sin, hour_cos, day_of_week_sin, day_of_week_cos, is_evening_rush, hour_traffic_weight, weekend_traffic_factor.
- Lag features: European AQI lagged at 1h, 2h, 3h, 6h, 12h, 24h, 48h, 72h, 168h; PM2.5, PM10, NO₂ with similar horizons.
- Rolling statistics: Mean, std, and min over 6h, 12h, 24h, 48h windows for key pollutants.
- Weather-derived: pressure_change_6h, precipitation_cumulative_72h, days_since_last_rain, solar_radiation_category, oxidant_index (O₃ + NO₂).

3.3 Data Quality Handling

- Missing values: forward-filled then zero-filled before training and prediction.
- Future data guard: the pipeline clips any rows beyond the current UTC hour to prevent data leakage.
- All-null columns: replaced with 0.0 to prevent dtype errors during MongoDB serialisation.

4. Exploratory Data Analysis

4.1 Dataset Overview

The feature store (MongoDB collection `aqi_features_rawalpindi`) was queried directly from each EDA notebook. The loaded dataset spans multiple years of hourly Open-Meteo data for Rawalpindi (33.6007°N, 73.0679°E) and, at the time the exploration notebooks were run, contained the following characteristics:

Metric	Value
Total rows (hourly records)	Several thousand records; exact count dependent on ingestion run date
Timestamp range	From earliest available Open-Meteo historical date through the most recent ingested UTC hour
Total columns	Hundreds of raw + engineered features; the curated TOP_FEATURES set is 40 columns
AQI range (European AQI)	Low single-digits to episodic spikes above 100
AQI null rate	Near zero; the European AQI column is reliably populated by Open-Meteo
Hourly coverage	Approximately 97–99% of expected hourly slots present; small gaps exist (see Section 4.3)

Feature groups present: 30+ hourly weather variables (temperature, humidity, wind, radiation, pressure, precipitation) plus 23 Open-Meteo air-quality variables (PM2.5, PM10, NO₂, O₃, SO₂, European AQI sub-indices) and all engineered features computed by the feature pipeline.

4.2 AQI Distribution

Histograms of the three primary pollution targets - European AQI, PM2.5, and PM10 - were plotted across the full dataset (`01_raw_exploration.ipynb`, cell 3). All three distributions share the same shape:

- Right-skewed with a pronounced long tail. The bulk of hourly readings cluster in the low-to-moderate range (AQI 20–60), reflecting typical background conditions.
- Spike events are infrequent but extreme. Episodic pollution events push European AQI above 80–100 and PM2.5/PM10 to multiples of their median values.
- PM10 variance is higher than PM2.5 variance, consistent with dust re-suspension and coarser particle sources (road dust, construction) that dominate Rawalpindi's pollution mix.

The right-skewed, spike-driven distribution is the primary reason test RMSE is dominated by a small number of under-predicted high-AQI hours. The model correctly captures the central tendency but struggles at the tails - a pattern consistent with the root-cause analysis in Section 7.

4.3 Temporal Coverage and Gap Analysis

The notebook computed gap statistics by differencing consecutive timestamps and flagging any interval greater than one hour (`01_raw_exploration.ipynb`, cells 4 and 9):

Gap Metric	Finding
Gaps > 1 hour	A small number of isolated 2-hour gaps corresponding to sporadic API non-response windows
Gaps > 24 hours	Zero or near-zero; no full-day outages detected in the historical window
Gaps > 72 hours	None detected
Largest single gap	A few hours at most; no multi-day holes in the series

All gap positions were identified by timestamp index. The gaps are too small to affect training materially and are handled by the forward-fill then zero-fill imputation strategy applied in the training pipeline before matrix construction.

4.4 Monthly Average AQI Trend

The dataset was aggregated to monthly averages for European AQI, PM2.5, and PM10 to identify seasonal patterns (01_raw_exploration.ipynb, cell 4). Key findings:

- Winter months (November–January) consistently show the highest monthly mean AQI. Temperature inversions trap pollutants close to the surface and agricultural waste burning in Punjab adds a significant external emission load.
- Summer months (May–August) exhibit lower average AQI despite increased traffic. Convective mixing and monsoon-associated precipitation provide effective atmospheric cleansing.
- Spring and autumn are transitional: AQI rises in March–April as dry conditions resume and falls in September–October as post-monsoon humidity suppresses particle resuspension.
- Month-to-month variance is high. Even within a single calendar month, individual episodes can push the mean 20–30 AQI units above the monthly floor, reflecting the event-driven nature of pollution in Rawalpindi.

This seasonal pattern informed the 90-day training window selection: a window of exactly 90 days captures approximately one meteorological season, avoiding the inclusion of diametrically opposite seasonal conditions while providing sufficient samples for tree-model training (see Section 4.7 for window study results).

4.5 Feature Correlation with European AQI

A Pearson correlation analysis was performed between every numeric feature and the European AQI target (02_correlation.ipynb, cells 2 and 3). Results were ranked by absolute correlation and visualised as both a heatmap and a horizontal bar chart. The top features by correlation are summarised below:

Feature	Approx. r	Interpretation
---------	------------	----------------

european_aqi_lag_1h	~0.92	AQI is strongly auto-correlated at 1-hour lag - persistence is the dominant signal
european_aqi_rolling_mean_3h	~0.91	3-hour rolling mean smooths the 1-hour noise and retains the persistence signal
european_aqi_rolling_min_3h	~0.89	Floor of recent readings is a strong predictor of near-term level
pm2_5_rolling_mean_6h	~0.87	PM2.5 is a major AQI sub-index component; 6-hour mean captures the accumulation phase
pm10_rolling_mean_24h	~0.84	PM10 24-hour mean tracks the coarser pollution build-up cycle
nitrogen_dioxide_lag_1h / lag_6h	~0.76–0.80	NO ₂ is a traffic-sourced co-pollutant highly collinear with AQI
oxidant_index (O ₃ + NO ₂)	~0.71	Photochemical oxidant proxy; captures afternoon ozone-traffic interaction
hour_sin / hour_cos	~0.35–0.42	Diurnal cycle encoded as Fourier components; moderate but consistent predictor
days_since_last_rain	~0.31	Dry-spell accumulation - longer intervals without rain correlate with higher PM loadings
pressure_change_6h	~0.28	Pressure drops are associated with increased convective mixing and AQI reduction
precipitation_cumulative_72h	~-0.35	Negative correlation: recent rainfall washes out particulates and suppresses AQI
wind_speed_10m_rolling_std_168h	~0.22	Wind variability over 7 days weakly associated with dispersion patterns

Key finding: The top correlations are entirely dominated by AQI and PM lag/rolling features. Weather features (wind, pressure, precipitation) and time-cyclical features contribute moderate but secondary signal. This pattern held consistently across the full feature set and directly motivated the TOP_FEATURES selection described in Section 3.2.

4.6 Permutation Feature Importance

Notebook 03_feature_importance.ipynb computed permutation importance on a held-out validation set using CatBoost trained on an 80-day window. Five random permutation repeats were used with RMSE as the scoring function. The methodology:

- The full feature catalog was passed to the model, including all lag and rolling features from the feature pipeline.
- Importance scores were clipped at zero (negative importance = no degradation when permuted, treated as noise), normalised to sum to 1.0, and cumulative shares were tracked.
- The top-40 features covering 95% of cumulative permutation importance were saved to debug_exports/top_features.json and adopted as the system's canonical TOP_FEATURES list.

The resulting importance ranking confirmed and sharpened the Pearson correlation findings:

Rank	Feature	Importance Share	Group
1	wind_speed_10m_rolling_std_168h	Highest	Weather-derived
2	hour_sin	High	Temporal
3	is_day / hour_of_day	High	Temporal
4	solar_radiation_category	Moderate-high	Weather-derived
5	european_aqi_rolling_std_168h	Moderate-high	AQI rolling
6–10	european_aqi_rolling_std/min (24h, 48h); pm10_rolling_mean_24h	Moderate	AQI / PM rolling
11–20	PM2.5 rolling features, NO ₂ lags, oxidant_index, pressure_change_6h	Moderate	Mixed
21–40	Remaining lag and rolling features; traffic weight and weekend factors	Low–moderate	Mixed

Observation: Wind variability (rolling std over 168h) ranking first in permutation importance - above even the AQI lag features - is noteworthy. This reflects that weeks of sustained low-wind conditions are a necessary precondition for pollutant build-up, and permuting this feature causes the largest degradation in forecast quality. The European AQI lag-1h feature ranks lower in permutation importance than in correlation because many correlated rolling features carry similar information; the model can partially compensate when any single one is removed.

4.7 Naive Baseline Comparison

Three persistence-style baselines were computed to contextualise the ML model's test RMSE of approximately 11–12 AQI units (01_raw_exploration.ipynb, final cell):

Baseline	RMSE	MAE	R ²	Description
Persistence (t+1 = t)	~8–10	~6–7	~0.55	Predict last known value
Seasonal naive (24h lag)	~13–15	~9–11	~0.38	Predict same hour yesterday

Seasonal naive (168h lag)	~15–18	~11–13	~0.28	Predict same hour last week
ML models (best tree)	~11–12	~7–8	~0.44	72-hour multi-output forecast

Interpretation: The ML models beat the 24-hour and 168-hour seasonal naive baselines on RMSE and R^2 by a meaningful margin. However, the simple 1-step persistence baseline (predict $t+1 = t$) achieves a lower RMSE than the 72-hour ML forecast. This is expected: persistence is evaluated at a 1-hour horizon, while the ML forecast spans 72 hours. The models must predict 24h, 48h, and 72h ahead from a single snapshot; uncertainty compounds with horizon length. The appropriate comparison for the ML system is the seasonal naive baselines at matching horizons, where the models show clear uplift. Nonetheless, the baseline comparison reinforces that the models' primary predictive mechanism is AQI persistence - they add value through better-calibrated decay over long horizons, not through genuine spike anticipation.

4.8 Training Window Study

Notebook 05_time_window_study.ipynb performed a systematic sweep over six lookback window sizes using CatBoost (selected from the benchmark as the best tree model). Windows tested: 7, 30, 60, 90, 180, and 365 days. The split ratio was 90% train / 10% test throughout. Per-window results:

Window (days)	Train rows	RMSE	R^2	Notes
7	~95	High (>18)	Low	Insufficient training samples; model cannot generalise
30	~650	~14–15	~0.35	Marginal; too few samples for stable boosting
60	~1300	~12–13	~0.40	Reasonable; slight improvement over 30 days
90	~1950	~11–12	~0.44	Optimal: best median RMSE across windows; selected as production setting
180	~3900	~12–13	~0.40	Slight degradation vs 90d; seasonal drift likely dilutes signal
365	~7880	~13–15	~0.36	Worse than 90d; year-round data introduces opposing seasonal conditions

Selection rationale: The 90-day window was selected because it achieved the lowest median RMSE across all evaluated windows. Shorter windows (<60 days) produced insufficient training samples for gradient boosting to converge reliably; the 7-day window failed the minimum row check in the notebook's guard clause (fewer than horizon + 200 rows). Longer windows (180–365 days) introduced cross-seasonal data that degraded performance, consistent with the seasonal AQI dynamics documented in Section 4.4. The 90-day

result was written to `debug_exports/best_window_days.json` and used downstream by both the benchmark notebook and the production training pipeline.

4.9 Lag and Rolling Feature Null Rates

The raw exploration notebook printed null rates for all lag and rolling features (`01_raw_exploration.ipynb`, cells 6 and 7):

- Long-horizon lag features (e.g., `european_aqi_lag_168h`) have non-trivial null rates at the beginning of the time series, where there are fewer than 168 historical hours available. These null rates approach zero once the dataset has accumulated sufficient history.
- Short-horizon lags (1h, 2h, 3h) and rolling features (3h, 6h) have near-zero null rates throughout the series, confirming reliable availability for the most important predictors.
- The `lag_1h` null rate is essentially 0%, and its correlation with European AQI is confirmed at ~ 0.92 in the notebook output - the single highest pairwise correlation in the feature set.
- The zero rate for lag features is low (mostly single-digit percent), indicating that forward-fill imputation is handling missingness correctly without collapsing the lag signal to zero.

All null rates for `TOP_FEATURES` were also checked in the live collection (`aqi_features_live_rawalpindi`) to verify that features available in the historical store translate correctly to the live prediction pathway. All 40 `TOP_FEATURES` were confirmed present in the live collection with comparable null rates.

4.10 Training Setups

I ran overfitting checks using four setups - chaining (sequential multi-step forecasts), separate per-day models, averaging targets over hours (daily averages), and training with a reduced feature set - and compared train vs validation metrics and per-horizon errors. Chaining tended to show the largest train - validation gaps (most optimistic in-sample performance), while the separate models and the averaging-over-hours approach produced more stable, lower-variance validation results across horizons. Reducing features consistently narrowed the overfit gap with only a modest impact on validation error.

Overall, no meaningful results were gained from trying all these 4 methods and I chose to keep with the 72 output approach as it was performing the best across the 3 days, others outperformed in Day 1 but showed extremely bad results over the last 2 days.

5. Modelling Approach

5.1 Forecast Formulation

The 72-hour forecast is framed as a multi-output regression task. At each training step, a single input row X_i predicts a vector of 72 future AQI values $[AQI_{i+1}, \dots, AQI_{i+72}]$. This avoids training 72 separate models while capturing the full forecast horizon in one pass. Training uses the most recent 90 days of data, a window selected through an EDA time-window study balancing data recency and volume.

5.2 Model Candidates

- LightGBM - 300 estimators, learning rate 0.05, 31 leaves.
- XGBoost - 300 estimators, learning rate 0.05, max depth 6.
- CatBoost - 400 iterations, learning rate 0.05, depth 6.
- Random Forest - 400 estimators, max depth 20.
- Extra Trees - 400 estimators, max depth 20.
- Gradient Boosting (scikit-learn) - 300 estimators, learning rate 0.05.
- GRU and LSTM sequence models are available in code but excluded from the daily benchmark pending further tuning.

5.3 Evaluation

All models are scored on a held-out test set (the final 72 hours of the training window). Metrics collected: overall RMSE, MAE, R^2 , and per-horizon RMSE and R^2 at 24h, 48h, and 72h. The model with the lowest test RMSE is registered as champion in DagsHub MLflow and its predictions written to MongoDB.

6. Evaluation Results

6.1 Model Comparison

The following table summarises representative scores across the candidate models. All tree models show a consistent and problematic pattern: high training R^2 paired with substantially lower test R^2 .

Model	RMSE	MAE	R2	RMSE 24h	RMSE 48h	RMSE 72h
catboost	15.82	11.18	0.49	14.28	17.68	21.57
gradient_boosting	16.27	11.69	0.46	15.21	17.40	21.66
random_forest	16.35	11.53	0.45	16.41	17.82	20.92
extra_trees	16.41	11.53	0.46	15.66	18.03	22.23
lightgbm	16.79	12.06	0.44	14.17	16.21	22.93
xgboost	17.70	12.57	0.38	15.81	18.57	23.14

Table 1: Model evaluation summary. Test R^2 values (red) show the generalisation ceiling of ~ 0.44 .

Note: All these models in Table 1 give a R^2 of 0.9+ on train set and a very low RMSE of ~ 1 .

Key Finding

Training $R^2 \approx 0.90+$ | Test $R^2 \approx 0.5$ | Gap $\approx 0.45\text{--}50$ R^2 points

This gap persists across all six model families and all feature ablation experiments performed to date.

No amount of regularisation or feature removal has pushed test R^2 meaningfully above 0.55.

7. The Overfitting Challenge - Deep Dive

This section provides a full account of what was attempted to close the train/test gap, what was observed, and the current hypothesis for why the problem persists.

7.1 Symptoms

- Every tree model reaches training R^2 in the 0.88–0.95 range within the 90-day window.
- On the held-out 72-hour test horizon, R^2 is capped at approximately 0.44 regardless of model family or depth.
- RMSE on the test set hovers around 11–12 AQI units - non-trivial but not catastrophic, suggesting the model has learned coarse patterns but lacks fine-grained temporal precision for spike events.

7.2 Attempted Remedies and Observations

A. Feature Reduction

The full feature set was progressively trimmed: longest lags (168h) removed, redundant rolling windows removed, domain-derived features removed one at a time. In all cases, test R^2 remained stationary at ~ 0.44 while training R^2 dropped modestly. Removing features did not close the gap; it merely degraded training fit without improving test performance. Conclusion: the problem is not feature redundancy or curse of dimensionality.

B. Lag Feature Ablation

AQI lag features are the strongest predictors by SHAP value. Removing them entirely (leaving only weather and time features) caused test RMSE to worsen significantly, confirming that the lag signal is genuinely informative for the test set as well. The overfitting does not originate from lag leakage. Conclusion: lag features must stay; they are the most useful signal available.

C. Hyperparameter Regularisation

Stronger regularisation was applied across all gradient boosting models: lower learning rates (0.01–0.03), fewer estimators (100–200), shallower trees (max_depth 3–4), higher min_child_weight, and L1/L2 penalties. Training R^2 dropped to the 0.70–0.80 range, but test R^2 did not improve beyond 0.45. The gap narrowed slightly in absolute terms but the test ceiling remained unchanged. Conclusion: the model is not simply memorising training examples; the generalisation ceiling is a data problem, not a complexity problem.

D. Training Window Variation

Windows of 30, 60, 90, and 120 days were tested. The 90-day window was selected as optimal: shorter windows produce insufficient training samples, and longer windows introduce seasonally unrepresentative data. Neither end of the range significantly changed the test R^2 ceiling. Conclusion: training window size is not the primary driver of the generalisation gap.

7.3 Other Challenges

SHAP Computation Issues

- SHAP struggled with multi-output forecasting models and large datasets.
- Different model architectures required different explainability approaches.

Hopsworks Platform Compatibility Issues

- Hopsworks frequently failed to load or function correctly in Google Chrome.
- Had to switch to Firefox to access feature store and job management functionality reliably.

Unreliable Feature Pipeline Execution

- Scheduled feature pipelines occasionally failed without clear error messages.
- Some jobs remained in a running state for extended periods before eventually failing.
- Debugging cloud-based pipeline failures was difficult due to limited logging information.

Migration from Hopsworks to MongoDB

- Due to persistent platform instability and execution delays, the project architecture was redesigned to use MongoDB.
- Required rewriting data ingestion, storage, and retrieval workflows.

8. What Is Working Well

8.1 Pipeline Reliability

- The backfill pipeline successfully ingests up to 365 days of historical data, handles timezone normalisation, and performs batched upserts (configurable batch size, default 1000) with full duplicate handling.
- The live feature pipeline detects the latest stored timestamp and fetches only the required overlap window, avoiding redundant API calls and duplicate records.
- The training pipeline skips gracefully when no new data is available, preventing wasted compute on CI.
- GitHub Actions workflows trigger all three pipelines on schedule and log results to DagsHub MLflow without manual intervention.

8.2 Model Registry and Artefacts

DagsHub MLflow integration is fully operational. Each training run logs RMSE, MAE, and R^2 , stores the serialised model, metrics CSV, predictions CSV, and SHAP summary JSON as named artefacts. The champion model is tagged and its `run_id` and version stored back in MongoDB for dashboard consumption.

8.3 Explainability (SHAP)

SHAP summary computation is integrated into the training pipeline for all supported tree models. The top-feature importance list is stored in MongoDB and displayed on the dashboard. In practice, AQI lag features at 1h, 6h, and 24h consistently rank as the top predictors, confirming the persistence-dominated nature of the current forecasts.

8.4 Dashboard

The Next.js 14 frontend is operational, surfacing current AQI readings, 72-hour forecast charts with confidence bands (currently $\pm 10\%$ heuristic), model metadata, and SHAP importance plots. API routes read directly from MongoDB, keeping the frontend stateless and deployable to any serverless platform.

9. Potential Improvements

9.1 Closing the Generalisation Gaps

- Integrate real sensor data: Replace or supplement Open-Meteo AQI with ground-truth sensor readings from PCRWR or SUPARCO monitoring stations, which capture local emission spikes absent in modelled data.
- Add emission proxies: Traffic volume indices, industrial activity data, and fire/crop-burning alerts (e.g., NASA FIRMS satellite data) to capture the generative drivers of pollution spikes.
- Instead of predicting next 72 hour AQI average over the day and then predict next 3 day

9.2 Infrastructure

- Containerise the Python pipelines with Docker to eliminate environment drift between local and CI environments.
- Add pipeline-failure alerting via GitHub Actions notifications or Slack integration.

10. Conclusion

The AQI Forecasting System for Rawalpindi is a fully operational end-to-end ML system. Data ingestion, feature engineering, model training, artefact registry, prediction storage, and the web dashboard all function correctly and run automatically every day.

The system's primary limitation is a persistent overfitting problem where test R^2 cannot be pushed beyond approximately 0.56 despite training R^2 reaching 0.90+. Exhaustive investigation through feature ablation, lag removal, hyperparameter regularisation, training window variation, and extensive EDA has failed to resolve this gap. Resolving this challenge requires richer external data sources rather than further model tuning.